

D3.2 Software Engineering Standards Manual

Due date: **15/03/2026**
Submission Date: **15/03/2026**
Revision Date: **15/08/2026**

Start date of project: **01/07/2026**

Duration: **12 months**

Responsible Person: **Yohannes Haile**

Revision: **1.3**

Project funded by Upanzi Network Dissemination Level		
PU	Public	PU
PP	Restricted to other programme participants	
RE	Restricted to a group specified by the consortium	
CO	Confidential, only for members of the consortium	

Executive Summary

Deliverable D3.2 presents the Software Engineering Standards Manual for the DEC Pepper Robot Tour project. This manual defines the software engineering standards and practices to be followed by all contributors developing software for the Pepper robot's autonomous tour guide system. It specifies: the guiding principles based on Component-Based Software Engineering (CBSE) applied to ROS2; the software development environment including tools, libraries, and version control; and the standards for specification, design, implementation, testing, and documentation of all software modules.

The manual is intended to ensure that all software developed within WP4 (Robot Sensing) and WP5 (Robot Behaviors) is consistent, interoperable, maintainable, and well-documented, enabling successful system integration as described in Deliverable D3.3.

Contents

I	Guiding Principles	5
1	Introduction	5
1.1	Scope	5
1.2	Overview	5
2	Component-Based Software Engineering	5
2.1	The Component-Port-Connector Model	5
2.2	Guiding Principles	6
II	Software Development Environment	7
3	Development Platform	7
3.1	ROS2 Framework	7
3.2	Build System	7
3.3	Version Control and Repository	7
III	Standards	8
4	Specification Standard	8
4.1	Module Specification	8
4.2	Interface Specification	8
5	Design Standard	8
5.1	Package Structure Design	8
5.2	Node Design Pattern	9
6	Implementation Standard	9
6.1	Python Implementation	9
6.2	C++ Implementation	9
7	Documentation Standard	10
7.1	Required Documentation	10
7.2	Deliverable Report Standard	10
Appendix A	File Organization	11
A.1	Workspace Layout	11
A.2	Python Package Structure	11
A.3	C++ Package Structure	12
A.4	Filename Conventions	12
A.5	Package Build Files	12
A.5.1	setup.py – Python Package	12
A.5.2	package.xml – Python Package	13
A.5.3	Python Launch File	14

A.5.4	YAML Configuration File	14
Appendix B	Documentation	15
B.1	Source File Header	15
B.1.1	Python File Header	15
B.1.2	C++ File Header	15
B.2	Application File API Documentation	15
B.3	Implementation File Documentation	17
B.3.1	Python Class and Method Docstrings	17
B.3.2	C++ Doxygen Comments	18
Appendix C	Programming Style	19
C.1	Naming Conventions	19
C.2	Import and Include Ordering	19
C.2.1	Python Import Order	19
C.2.2	C++ Include Order	20
Appendix D	Programming Practice	21
D.1	Python Best Practices	21
D.1.1	Type Hints	21
D.1.2	ROS2 Parameter Declaration	21
D.1.3	ROS2 Logging	22
D.2	C++ Best Practices	22
D.2.1	Header Guards	22
D.2.2	Smart Pointers	22
D.2.3	ROS2 Logging Macros	23
D.2.4	ROS2 Parameter Declaration in C++	23
Principal Contributors		25
Document History		26

Part I

Guiding Principles

1 Introduction

This document provides the software engineering standards manual for the Pepper Robot Tour for Upanzi's Digital Experience Center (DEC). The purpose is to establish a common set of engineering standards and practices for the development of software to be used in the DEC project, based on the Robot Operating System 2 (ROS2) Humble Hawksbill Open Robotics (2022) running on Ubuntu 22.04 LTS.

The software engineering standards are derived from established best practices in component-based software engineering Szyperski (2002) and are adapted to the specific requirements of ROS2 robotics software development. The standards are mandatory for all software submitted for integration into the DEC system.

1.1 Scope

These standards apply to all software modules developed in:

- **WP4 – Robot Sensing:** Person detection, face detection and gaze estimation, speech event detection, robot localization, conversation manager.
- **WP5 – Robot Behaviors:** ROS2 actuator control, behavior controller, Nav2 navigation, gesture animation system.

1.2 Overview

The DEC software follows a component-based architecture built on ROS2. Each software component is realized as a ROS2 *node* and communicates with other components using ROS2 topics, services, and actions. The system is organized as a *meta-package* (`dec_system`) containing individual functional packages, each implementing a specific sensing or behavior capability of the Pepper robot.

2 Component-Based Software Engineering

The DEC software is developed using Component-Based Software Engineering (CBSE) Szyperski (2002). In CBSE, a software system is composed of *components* — independently deployable units of software that interact through well-defined *interfaces*. CBSE promotes reusability, modularity, and replaceability of software units.

2.1 The Component-Port-Connector Model

The DEC software adopts the Component-Port-Connector (CPC) meta-model, in which:

- A **component** is a ROS2 node encapsulating a specific robot capability (e.g., face detection, behavior control).
- A **port** is a ROS2 topic, service, or action interface through which a component communicates.

- A **connector** is the ROS2 publish-subscribe or request-response mechanism linking producer ports to consumer ports.

Each component has:

- **Input ports** — topics subscribed to (consumed data) or services/actions provided.
- **Output ports** — topics published (produced data) or services/actions requested.
- **Configuration** — parameters loaded from YAML configuration files.

2.2 Guiding Principles

The following guiding principles apply to all DEC software development:

1. **Component Autonomy:** Each ROS2 node must be independently executable and testable using a driver-stub approach, without requiring other system components to be running.
2. **Interface Stability:** Published topic names, message types, service names, and action names constitute the public interface of a node and must not change without a version increment.
3. **Configuration-Driven Behavior:** All tunable parameters must be exposed via the ROS2 parameter server and loaded from a dedicated configuration file. Hard-coded parameter values are prohibited.
4. **Separation of Application and Implementation:** Each ROS2 package must separate the entry-point logic (application file) from the node implementation (implementation file), following the two-file pattern defined in Appendix A.
5. **Explicit Dependency Declaration:** All ROS2 and third-party library dependencies must be declared in `package.xml` and, for Python packages, in `setup.py`.
6. **Reproducible Build:** All packages must build successfully from source using `colcon build --symlink-install` in the `dec_ws` workspace without manual intervention.
7. **Naming Convention Compliance:** All file names, package names, node names, topic names, and parameter names must comply with the naming conventions defined in Appendix C.
8. **Mandatory Documentation:** Every source file must carry the standard file header. Every public function, class, and method must carry an API documentation comment as specified in Appendix B.
9. **Version Control:** All source code, configuration files, launch files, and models must be committed to the project GitHub repository at <https://github.com/yohatad/pepper4dec> with descriptive commit messages.

Part II

Software Development Environment

3 Development Platform

All DEC software is developed and deployed on **Ubuntu 22.04 LTS (Jammy Jellyfish)**. This is the officially supported platform for ROS2 Humble Hawksbill. Two programming languages (C++ and Python) will be used throughout the project.

3.1 ROS2 Framework

The Robot Operating System 2 (ROS2) **Humble Hawksbill** Open Robotics (2022) is the middleware framework for all DEC software. ROS2 provides:

- A publish-subscribe communication framework using DDS (Data Distribution Service) Object Management Group (2015).
- Client libraries: `rclpy` (Python) and `rclcpp` (C++).
- Build system: `ament_python` (Python packages) and `ament_cmake` (C++ packages).
- Build tool: `colcon` replacing ROS1's `catkin_make`.
- Launch system: Python `.launch.py` files.
- Parameter system with YAML configuration support.

3.2 Build System

Package Type	Build System	Build Files
Python node	<code>ament_python</code>	<code>setup.py</code> , <code>setup.cfg</code> , <code>package.xml</code>
C++ node	<code>ament_cmake</code>	<code>CMakeLists.txt</code> , <code>package.xml</code>
Interface package	<code>ament_cmake</code>	<code>CMakeLists.txt</code> , <code>package.xml</code>

Table 1: Build systems for different package types.

The standard build command for the entire workspace is:

```
cd ~/dec_ws
colcon build --symlink-install
source install/setup.bash
```

3.3 Version Control and Repository

All DEC software is hosted on GitHub at:

<https://github.com/yohatad/pepper4dec>

The repository contains the complete `dec_ws` workspace source. Contributors must fork the repository and submit pull requests for integration. Direct pushes to the `main` branch are not permitted.

Part III

Standards

4 Specification Standard

4.1 Module Specification

Every ROS2 node must be specified before implementation begins. The specification must include:

1. **Purpose:** A concise statement of what the node does.
2. **Input Ports:** All subscribed topics with message types and expected publish rates.
3. **Output Ports:** All published topics with message types and publish rates.
4. **Services:** Any ROS2 services provided or called by the node.
5. **Actions:** Any ROS2 actions provided or used by the node.
6. **Parameters:** All configuration parameters with names, types, default values, and valid ranges.
7. **Dependencies:** All ROS2 packages and third-party libraries required.

4.2 Interface Specification

Custom message types are defined in the `dec_interfaces` package. Interface definitions must follow ROS2 `.msg`, `.srv`, and `.action` file format conventions.

5 Design Standard

5.1 Package Structure Design

Each DEC node must be designed as an independent ROS2 package adhering to the standard package structure defined in Appendix A. The design must clearly distinguish:

- The **application file** — the entry point that initializes `rclpy/rclcpp`, instantiates the node, and handles lifecycle.
- The **implementation file** — the ROS2 node class implementing all publishers, subscribers, callbacks, and computational logic.
- The **configuration file** — YAML file containing all tunable parameters.
- The **launch file** — Python `.launch.py` file for launching the node.

5.2 Node Design Pattern

Each package's primary node must be a ROS2 *lifecycle* node, following the *callback-based* ROS2 design pattern:

1. The constructor declares parameters only; publishers, subscribers, timers, and clients are created in `on_configure()`, so construction stays cheap and side-effect-free.
2. `on_configure()`, `on_activate()`, `on_deactivate()`, `on_cleanup()`, and `on_shutdown()` lifecycle transition callbacks acquire and release resources (models, connections, threads) at the appropriate stage, driven by a lifecycle manager (see `dec_launch`) rather than by the node itself.
3. Callback functions process incoming messages and trigger computation.

Secondary, non-orchestrated helper nodes within a package (e.g., a visualization or debug-only node) may use a plain node instead.

6 Implementation Standard

6.1 Python Implementation

Python nodes must:

- Use `rclpy.lifecycle.LifecycleNode` as the base class for a package's primary node (`rclpy.node.Node` for secondary helper nodes).
- Declare all parameters using `self.declare_parameter()` in the constructor.
- Use Python type hints for all function signatures.
- Follow PEP 8 style conventions van Rossum et al. (2001) (enforced by `flake8`).
- Use the logging macros `self.get_logger().info()`, `.warning()`, and `.error()` for all output (no `print()` statements).

6.2 C++ Implementation

C++ nodes must:

- Use `rclcpp_lifecycle::LifecycleNode` as the base class for a package's primary node (`rclcpp::Node` for secondary helper nodes).
- Declare all parameters using `this->declare_parameter()` in the constructor.
- Use `RCLCPP_INFO`, `RCLCPP_WARN`, and `RCLCPP_ERROR` macros for all logging.
- Follow the Google C++ Style Guide Google LLC (2024) (enforced by `ament_cpplint`).
- Use smart pointers (`std::shared_ptr`, `std::weak_ptr`) for all heap-allocated objects.
- Use `#pragma once` header guards in all header files.

7 Documentation Standard

7.1 Required Documentation

Each software module must provide:

1. **Source file headers** — as specified in Appendix B.
2. **API documentation comments** — for all public functions and classes.
3. **Configuration file documentation** — inline comments explaining each parameter.
4. **README** — installation, build, and usage instructions specific to the package.
5. **Deliverable report section** — formal description of the module for the technical deliverable document.

7.2 Deliverable Report Standard

Each deliverable report section must include:

- A component specification table (ports, parameters, dependencies).
- A description of the algorithms and methods used.
- Experimental results or test results.
- References to relevant literature.

Appendix A File Organization

A.1 Workspace Layout

All DEC software resides in the `dec_ws` ROS2 workspace with the following structure:

```
dec_ws/
├── src/
│   ├── dec.system/
│   │   ├── animate_behavior/
│   │   ├── behavior_controller/
│   │   ├── conversation_manager/
│   │   ├── dec_common/
│   │   ├── dec_interfaces/
│   │   ├── dec_launch/
│   │   ├── face_detection/
│   │   ├── gesture_execution/
│   │   ├── overt_attention/
│   │   ├── pepper_navigation/
│   │   ├── pepper_slam/
│   │   ├── person_detection/
│   │   ├── speech_event/
│   │   └── text_to_speech/
```

A.2 Python Package Structure

A Python ROS2 package (e.g., `text_to_speech`) must follow this structure:

```
text_to_speech/
├── text_to_speech/
│   ├── __init__.py
│   ├── text_to_speech.application.py
│   └── text_to_speech.implementation.py
├── launch/
│   └── text_to_speech.launch.py
├── config/
│   └── text_to_speech.configuration.yaml
├── data/
│   └── pepper_topics.yaml
├── scripts/
│   └── text_to_speech
├── resource/
│   └── text_to_speech
├── package.xml
├── setup.py
└── setup.cfg
```

Packages that bundle trained models or unit tests add `models/` and `test/` directories following the same pattern.

A.3 C++ Package Structure

A C++ ROS2 package (e.g., behavior_controller) must follow this structure:

```
behavior_controller/
├── include/
│   ├── behavior_controller/
│   │   └── behavior_controller_implementation.h
├── src/
│   ├── behavior_controller_application.cpp
│   └── behavior_controller_implementation.cpp
├── launch/
│   └── behavior_controller.launch.py
├── config/
│   └── behavior_controller_configuration.yaml
├── missions/
│   └── dec_tour_mission.xml
├── test/
│   └── test_behavior_controller.cpp
├── package.xml
└── CMakeLists.txt
```

A.4 Filename Conventions

File Type	Convention	Example
Python application	<nodeName>_application.py	text_to_speech_application.py
Python implementation	<nodeName>_implementation.py	text_to_speech_implementation.py
C++ application	<nodeName>_application.cpp	behavior_controller_application.cpp
C++ implementation	<nodeName>_implementation.cpp /.h	behavior_controller_implementation.cpp
YAML config	<nodeName>_configuration.yaml	text_to_speech_configuration.yaml
Launch file	<nodeName>.launch.py	text_to_speech.launch.py

Table 2: Filename conventions for DEC packages.

A.5 Package Build Files

A.5.1 setup.py – Python Package

The setup.py file for a Python package must follow this template, shown for the text_to_speech package:

```
1 from setuptools import setup
2 from glob import glob
3
4 pkg = "text_to_speech"
5
6 setup(
```

```

7     name=pkg,
8     version="0.1.0",
9     packages=[pkg],
10    maintainer="DEC Project",
11    maintainer_email="dec-pepper-tour@andrew.cmu.edu",
12    license="BSD-3-Clause",
13    entry_points={
14        "console_scripts": [
15            "text_to_speech = text_to_speech.text_to_speech_application:
16            main",
17        ],
18    },
19    data_files=[
20        ("share/ament_index/resource_index/packages", [f"resource/{pkg}"
21        ]),
22        (f"share/{pkg}", ["package.xml"]),
23        (f"share/{pkg}/launch", glob("launch/*.launch.py")),
24        (f"share/{pkg}/config", glob("config/*")),
25        (f"share/{pkg}/data", glob("data/*")),
26    ],
27 )

```

Listing 1: setup.py for the text_to_speech package.

A.5.2 package.xml – Python Package

The package.xml (format 3) for the text_to_speech package:

```

1 <?xml version="1.0"?>
2 <?xml-model href="http://download.ros.org/schema/package_format3.xsd"?>
3 <package format="3">
4   <name>text_to_speech</name>
5   <version>0.1.0</version>
6   <description>
7     ROS2 node for text-to-speech synthesis and playback
8     on the Pepper robot.
9   </description>
10  <maintainer email="dec-pepper-tour@andrew.cmu.edu">DEC Project</
11  maintainer>
12  <license>BSD-3-Clause</license>
13
14  <buildtool_depend>setuptools</buildtool_depend>
15
16  <exec_depend>rclpy</exec_depend>
17  <exec_depend>std_msgs</exec_depend>
18  <exec_depend>naoqi_bridge_msgs</exec_depend>
19  <exec_depend>dec_interfaces</exec_depend>
20
21  <export>
22    <build_type>ament_python</build_type>
23  </export>
24 </package>

```

Listing 2: package.xml for the text_to_speech package.

A.5.3 Python Launch File

The ROS2 launch file for the `text_to_speech` node:

```

1 import os
2 from launch import LaunchDescription
3 from launch_ros.actions import Node
4 from ament_index_python.packages import get_package_share_directory
5
6 def generate_launch_description():
7     config = os.path.join(
8         get_package_share_directory('text_to_speech'),
9         'config',
10        'text_to_speech_configuration.yaml'
11    )
12
13    return LaunchDescription([
14        Node(
15            package='text_to_speech',
16            executable='text_to_speech',
17            name='text_to_speech',
18            output='screen',
19            parameters=[config]
20        )
21    ])

```

Listing 3: Python launch file for the `text_to_speech` node.

A.5.4 YAML Configuration File

The YAML configuration file for the `text_to_speech` node:

```

1 text_to_speech:
2   ros__parameters:
3     # Synthesis/playback backend
4     engine: naoqi_ros                # naoqi_ros, kokoro_local,
5     kokoro_pepper, ...              # kokoro_pepper
6     naoqi_speech_topic: /speech      # Text topic for naoqi_bridge's on-
7     board TTS
8
9     # Debugging
10    verbose_mode: false               # Enable verbose console output
11
12    # Playback pacing
13    chars_per_second: 12.0            # Estimated speech rate, for
14    feedback timing
15    speech_padding_s: 0.5             # Extra pause appended after each
16    sentence
17
18    # Local synthesis settings (kokoro_local / kokoro_pepper)
19    voice: af_bella                   # Kokoro-82M voice preset
20    sample_rate: 24000                # Output sample rate, in Hz

```

Listing 4: YAML configuration file for the `text_to_speech` node.

Appendix B Documentation

B.1 Source File Header

Every source file must begin with a standard file header. The header must include: the filename, a one-line description, the author, affiliation, email, date, and version.

B.1.1 Python File Header

```
1 """ text_to_speech_application.py
2
3     ROS2 Node for Text-to-Speech Synthesis and Playback.
4
5     Author:      Yohannes Tadesse Haile
6     Affiliation:  Carnegie Mellon University Africa
7     Email:       dec-pepper-tour@andrew.cmu.edu
8     Date:        April 18, 2025
9     Version:     v1.0
10
11     Copyright (C) 2025 Carnegie Mellon University Africa
12     This software is provided 'as-is' for research and
13     educational purposes within the DEC project.
14 """
```

Listing 5: Standard Python source file header.

B.1.2 C++ File Header

```
1 /* behavior_controller_application.cpp
2  *
3  * ROS2 node for mission-based behavior execution using
4  * BehaviorTree.CPP for the DEC Pepper Tour.
5  *
6  * Author:      Yohannes Tadesse Haile
7  * Affiliation:  Carnegie Mellon University Africa
8  * Email:       dec-pepper-tour@andrew.cmu.edu
9  * Date:        April 18, 2025
10 * Version:     v1.0
11 *
12 * Copyright (C) 2025 Carnegie Mellon University Africa
13 * This software is provided 'as-is' for research and
14 * educational purposes within the DEC project.
15 */
```

Listing 6: Standard C++ source file header.

B.2 Application File API Documentation

The application file (entry point) must document the node's complete interface in its module-level docstring. This includes all topics subscribed and published, with message types and topic names.

```

1  """ text_to_speech_application.py
2
3  ROS2 Node for Text-to-Speech Synthesis and Playback.
4
5  The text_to_speech node converts incoming text into spoken audio
6  and plays it through Pepper's speakers, using one of several
7  interchangeable synthesis backends.
8
9  Subscribers:
10     /text_to_speech/input      (std_msgs/String)
11         Sentences to speak, enqueued and played in order as they
12         arrive.
13
14  Publishers:
15     /text_to_speech/speaking   (std_msgs/Bool)
16         True while Pepper is actively speaking, false otherwise.
17
18  Parameters (declared as ROS2 parameters in the constructor;
19  overridable via text_to_speech_configuration.yaml at launch or
20  'ros2 param set' at runtime):
21     engine                     (str,      default: 'naoqi_ros')
22     naoqi_speech_topic         (str,      default: '/speech')
23     chars_per_second           (float,    default: 12.0)
24     speech_padding_s           (float,    default: 0.5)
25     voice                      (str,      default: 'af_bella')
26     sample_rate                (int,      default: 24000)
27
28  Author:      Yohannes Tadesse Haile
29  Affiliation:  Carnegie Mellon University Africa
30  Email:       dec-pepper-tour@andrew.cmu.edu
31  Date:        April 18, 2025
32  Version:     v1.0
33
34  """
35
36  import rclpy
37  from .text_to_speech_implementation import TextToSpeechNode
38
39  SOFTWARE_VERSION = "v1.0"
40
41  def main():
42      """Entry point for the text_to_speech ROS2 node.
43
44      Initializes rclpy, instantiates the TextToSpeechNode, and spins
45      until shutdown. The node starts in the UNCONFIGURED lifecycle
46      state; the on_configure()/on_activate() and
47      on_deactivate()/on_cleanup() transition callbacks (driven by a
48      lifecycle manager, not by this entry point) read the declared
49      parameters and acquire/release resources as the node is brought
50      up and torn down.
51      """
52      rclpy.init()
53      node = None
54      try:

```



```

53     node = TextToSpeechNode()
54     rclpy.spin(node)
55     except KeyboardInterrupt:
56         pass
57     finally:
58         if node is not None:
59             node.destroy_node()
60         if rclpy.ok():
61             rclpy.shutdown()
62
63 if __name__ == "__main__":
64     main()

```

Listing 7: Application file docstring with full API documentation.

B.3 Implementation File Documentation

Every class and public method in the implementation file must carry a docstring (Python) or Doxygen comment (C++).

B.3.1 Python Class and Method Docstrings

```

1 class TextToSpeechNode(LifecycleNode):
2     """ROS2 lifecycle node for text-to-speech synthesis and playback.
3
4     Subscribes to a text input topic and queues sentences for
5     sequential playback through one of several interchangeable
6     synthesis backends. Publishes a speaking-state flag to
7     /text_to_speech/speaking.
8
9     Attributes:
10         queue_ (queue.Queue): Pending sentences awaiting playback.
11         publisher_ (Publisher): Publisher for the speaking-state flag.
12         subscription_ (Subscription): Subscriber for text input.
13     """
14
15     def __init__(self) -> None:
16         """Declare parameters only; heavy setup is deferred to
17         on_configure().
18
19         Declaring parameters here (rather than reading a pre-loaded
20         config dict) means they survive repeated configure/cleanup
21         cycles and can be overridden via 'ros2 param set'.
22         """
23         super().__init__('text_to_speech')
24         self.declare_parameter('engine', 'naoqi_ros')
25         self.declare_parameter('voice', 'af_bella')
26         # ... remaining declare_parameter() calls ...
27
28     def on_configure(self, state: State) -> TransitionCallbackReturn:
29         """Read parameters and create the publisher, subscription, and
30         playback thread.

```

```

29
30     Args:
31         state (rclpy.lifecycle.State): The prior lifecycle state.
32         """
33         self.engine = self.get_parameter('engine').value
34         self.voice = self.get_parameter('voice').value
35         # ... remaining get_parameter() calls, then publisher_/
subscription_ creation ...
36         return TransitionCallbackReturn.SUCCESS
37
38     def text_callback(self, msg: String) -> None:
39         """Process an incoming text message.
40
41         Enqueues the sentence for playback by the background
42         playback thread, which drains the queue in order using
43         the configured synthesis backend.
44
45         Args:
46             msg (std_msgs/String): Incoming sentence to speak.
47             """
48         # ... callback implementation ...
49
50     def on_cleanup(self, state: State) -> TransitionCallbackReturn:
51         """Release all resources held by the node.
52
53         Stops the playback thread, closes the active audio
54         stream, and destroys the publisher and subscription.
55
56         Args:
57             state (rclpy.lifecycle.State): The prior lifecycle state.
58             """
59         # ... cleanup implementation ...
60         return TransitionCallbackReturn.SUCCESS

```

Listing 8: Python class and method documentation pattern.

B.3.2 C++ Doxygen Comments

```

1  /**
2   * @class BehaviorControllerNode
3   * @brief ROS2 lifecycle node for executing mission behavior trees.
4   *
5   * Loads a BehaviorTree.CPP XML mission file and executes the
6   * behavior tree in response to operator commands. Communicates
7   * with actuator control and navigation nodes via ROS2 topics
8   * and action servers.
9   */
10 class BehaviorControllerNode : public rclcpp_lifecycle::LifecycleNode {
11 public:
12     using CallbackReturn =
13         rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::
CallbackReturn;

```

```

14
15  /**
16   * @brief Construct a new BehaviorControllerNode.
17   *
18   * Declares all parameters. Loading the mission XML file,
19   * registering BehaviorTree.CPP nodes, and creating ROS2
20   * publishers and subscribers are deferred to on_configure().
21   */
22  explicit BehaviorControllerNode();
23
24  /** @brief Load the mission file and build the behavior tree. */
25  CallbackReturn on_configure(const rclcpp_lifecycle::State& state)
26  override;
27
28  /** @brief Start the behavior-tree tick timer. */
29  CallbackReturn on_activate(const rclcpp_lifecycle::State& state)
30  override;
31
32  /** @brief Cancel the tick timer; the tree stays built. */
33  CallbackReturn on_deactivate(const rclcpp_lifecycle::State& state)
34  override;
35
36  /**
37   * @brief Release all resources held by the node.
38   *
39   * Halts the running behavior tree, cancels pending action
40   * goals, and destroys the BehaviorTree.CPP factory.
41   */
42  CallbackReturn on_cleanup(const rclcpp_lifecycle::State& state)
43  override;
44
45  private:
46  /**
47   * @brief Callback triggered when a mission command is received.
48   *
49   * @param msg The mission command message containing the
50   *             mission name and execution parameters.
51   */
52  void missionCommandCallback(
53      const dec_interfaces::msg::MissionCommand::SharedPtr msg);
54  };

```

Listing 9: C++ Doxygen-style documentation pattern.

Appendix C Programming Style

C.1 Naming Conventions

C.2 Import and Include Ordering

C.2.1 Python Import Order

Python imports must be grouped in the following order, with a blank line between each group:

Entity	Python	C++	Example
Package name	snake_case	snake_case	text_to_speech
Node name	snake_case	snake_case	text_to_speech
Class name	PascalCase	PascalCase	TextToSpeechNode
Function/method	snake_case	camelCase	image_callback / imageCallback
Variable	snake_case	camelCase	confidence_threshold
Constant	UPPER_SNAKE	UPPER_SNAKE	SOFTWARE_VERSION
Private member	_name	name_	_session/session_
Topic name	snake_case/data	snake_case/data	/text_to_speech/spea king
Parameter name	snake_case	snake_case	confidence_threshold
File name	snake_case.py	snake_case.cpp/.h	text_to_speech_appli cation.py

Table 3: Naming conventions for DEC software.

1. Standard library modules (os, sys, time, etc.)
2. Third-party libraries (numpy, cv2, onnxruntime, etc.)
3. ROS2 libraries (rclpy, sensor_msgs, dec_interfaces, etc.)
4. Local package imports (relative imports using .)

```

1 # Standard library
2 import queue
3 import threading
4 from typing import Optional, List
5
6 # Third-party
7 import numpy as np
8
9 # ROS2
10 import rclpy
11 from rclpy.node import Node
12 from std_msgs.msg import String, Bool
13 from dec_interfaces.action import TTS
14
15 # Local (relative)
16 from .text_to_speech_implementation import TextToSpeechNode

```

Listing 10: Python import ordering example.

C.2.2 C++ Include Order

C++ includes must be grouped in the following order:

1. Corresponding header file (for .cpp files)
2. ROS2 headers (rclcpp, message types)
3. Third-party library headers (behaviortree_cpp_v3)

4. Standard library headers (string, memory, etc.)

```

1 // Corresponding header
2 #include "behavior_controller/behavior_controller_implementation.h"
3
4 // ROS2
5 #include "rclcpp/rclcpp.hpp"
6 #include "dec_interfaces/msg/mission_command.hpp"
7 #include "dec_interfaces/msg/mission_status.hpp"
8
9 // Third-party
10 #include "behaviortree_cpp_v3/bt_factory.h"
11 #include "behaviortree_cpp_v3/behavior_tree.h"
12
13 // Standard library
14 #include <string>
15 #include <memory>
16 #include <fstream>

```

Listing 11: C++ include ordering example.

Appendix D Programming Practice

D.1 Python Best Practices

D.1.1 Type Hints

All function signatures must use Python type hints. Return types must be annotated even for None-returning functions:

```

# Correct: full type annotation
def preprocess_image(self, image: np.ndarray,
                      size: tuple) -> np.ndarray:
    ...

def image_callback(self, msg: Image) -> None:
    ...

# Incorrect: missing type hints
def preprocess_image(self, image, size):
    ...

```

D.1.2 ROS2 Parameter Declaration

Parameters must be declared in the constructor, before the node is configured, so they survive repeated configure/cleanup cycles and can be overridden via `ros2paramset`. Retrieve values with `get_parameter().value` in `on_configure()`, not in the constructor:

```

def __init__(self) -> None:
    super().__init__('text_to_speech')

    # Declare parameters with defaults

```

```
self.declare_parameter('engine', 'naqi_ros')
self.declare_parameter('chars_per_second', 12.0)

def on_configure(self, state) -> TransitionCallbackReturn:
    # Retrieve parameter values
    self.engine = \
        self.get_parameter('engine').value
    self.chars_per_second = \
        self.get_parameter('chars_per_second').value
    # ... remaining setup ...
    return TransitionCallbackReturn.SUCCESS
```

D.1.3 ROS2 Logging

Use ROS2 logger macros exclusively. The `print()` function is prohibited in production code:

```
# Correct: use ROS2 logger
self.get_logger().info(f"Node initialized. Version: {SOFTWARE_VERSION}")
self.get_logger().warning("Image timeout -- no image received.")
self.get_logger().error(f"Failed to load ONNX model: {e}")

# Incorrect: do not use print()
print("Node initialized.")
```

D.2 C++ Best Practices

D.2.1 Header Guards

All C++ header files must use `#pragma once`:

```
#pragma once

#include "rclcpp_lifecycle/lifecycle_node.hpp"
#include "dec_interfaces/msg/mission_command.hpp"

class BehaviorControllerNode : public rclcpp_lifecycle::LifecycleNode {
    // ...
};
```

D.2.2 Smart Pointers

Heap-allocated objects must use smart pointers. Raw `new/delete` is prohibited:

```
// Correct: use smart pointers
auto node = std::make_shared<BehaviorControllerNode>();
std::unique_ptr<BT::BehaviorTreeFactory> factory_ =
    std::make_unique<BT::BehaviorTreeFactory>();

// Incorrect: raw new/delete
BehaviorControllerNode* node = new BehaviorControllerNode();
```

D.2.3 ROS2 Logging Macros

Use the appropriate RCLCPP_* macro for all logging:

```
// Info-level logging
RCLCPP_INFO(this->get_logger(),
    "BehaviorControllerNode initialized. Version: %s",
    SOFTWARE_VERSION);

// Warning-level logging
RCLCPP_WARN(this->get_logger(),
    "Mission XML file not found: %s", mission_file_.c_str());

// Error-level logging
RCLCPP_ERROR(this->get_logger(),
    "Failed to execute behavior tree: %s", e.what());
```

D.2.4 ROS2 Parameter Declaration in C++

Parameters must be declared with types and descriptions using ParameterDescriptor:

```
// Declare a string parameter with description
rcl_interfaces::msg::ParameterDescriptor mission_desc;
mission_desc.description = "Path to the BehaviorTree XML mission file";
this->declare_parameter<std::string>(
    "mission_file", "dec_tour_mission.xml", mission_desc);

// Retrieve parameter value
std::string mission_file_ =
    this->get_parameter("mission_file").as_string();
```

References

- Google LLC (2024). Google C++ style guide. <https://google.github.io/styleguide/cppguide.html>. Accessed: April 2025.
- Object Management Group (2015). Data Distribution Service (DDS) specification, version 1.4. Technical Report formal/2015-04-10, Object Management Group.
- Open Robotics (2022). ROS 2 Humble Hawksbill: Robot operating system 2. <https://docs.ros.org/en/humble/index.html>. Accessed: April 2025.
- Szyperski, C. (2002). *Component Software: Beyond Object-Oriented Programming*. Addison-Wesley / ACM Press, 2nd edition.
- van Rossum, G., Warsaw, B., and Coghlan, N. (2001). PEP 8 – style guide for Python code. <https://peps.python.org/pep-0008/>. Accessed: April 2025.

Principal Contributors

The main authors of this deliverable are as follows (in alphabetical order).

Yohannes Haile, Carnegie Mellon University Africa.

Document History

Version 1.0

Initial release.
Yohannes Haile.
15 March 2026.

Version 1.1

Corrected the Naming Conventions table to require `snake_case` for ROS2 node and topic names, matching actual practice across the `pepper4dec` codebase (verified against 60+ node names and all published topics), and fixed its internal contradiction with the Filename Conventions table over C++ file naming.

Replaced `face_detection` as the manual's flagship Python worked example – it is actually a C++ package – with `text_to_speech`, the only real Python packages being `conversation_manager`, `speech_event`, and `text_to_speech`; rewrote every affected Appendix A–C example accordingly.

Corrected the Python/C++ Implementation standards and worked examples to require `LifecycleNode` (not a plain `Node`) as each package's primary node base class, matching its lifecycle-manager-driven orchestration in `dec_launch`.

Updated the Workspace Layout tree to drop the deleted `pepper_navmap` package and add `dec_common`, `pepper_navigation`, and `pepper_slam`.

Yohannes Haile.
06 August 2026.

Version 1.2

Removed the personal maintainer email from the `setup.py/package.xml` worked examples, replacing it with a non-personal project placeholder (`dec-pepper-tour@andrew.cmu.edu`).

Removed the same personal email from the remaining Python/C++ file-header and docstring examples, and from the changelog's own wording, so no personal address appears anywhere in the document.

Synced the `TextToSpeechNode` worked examples with the current `text_to_speech` source after its constructor changed to declare parameters directly instead of taking a `config: dict` argument.

Yohannes Haile.
07 August 2026.

Version 1.3

Added a Principal Contributors section and renamed the “Version History” section to “Document History” to match the other DEC deliverables; both sections are now listed in the table of contents.

Yohannes Haile.
15 August 2026.